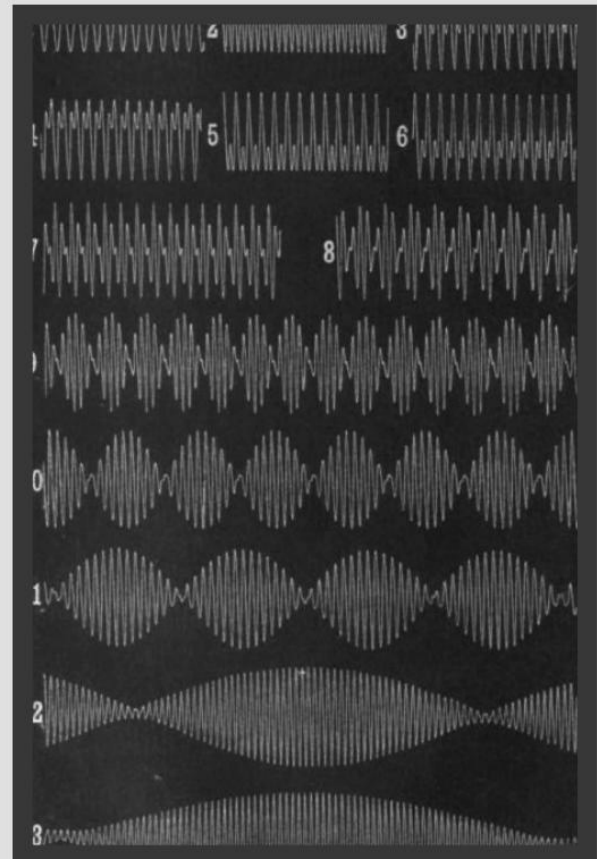
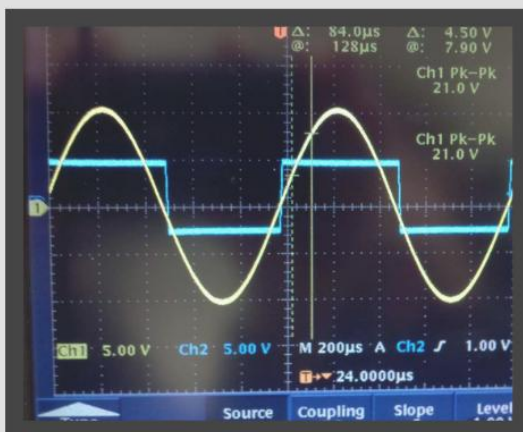
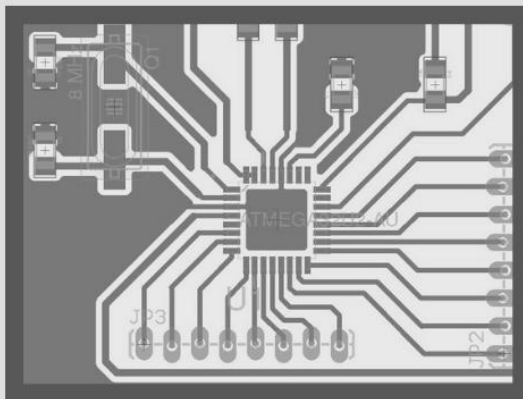
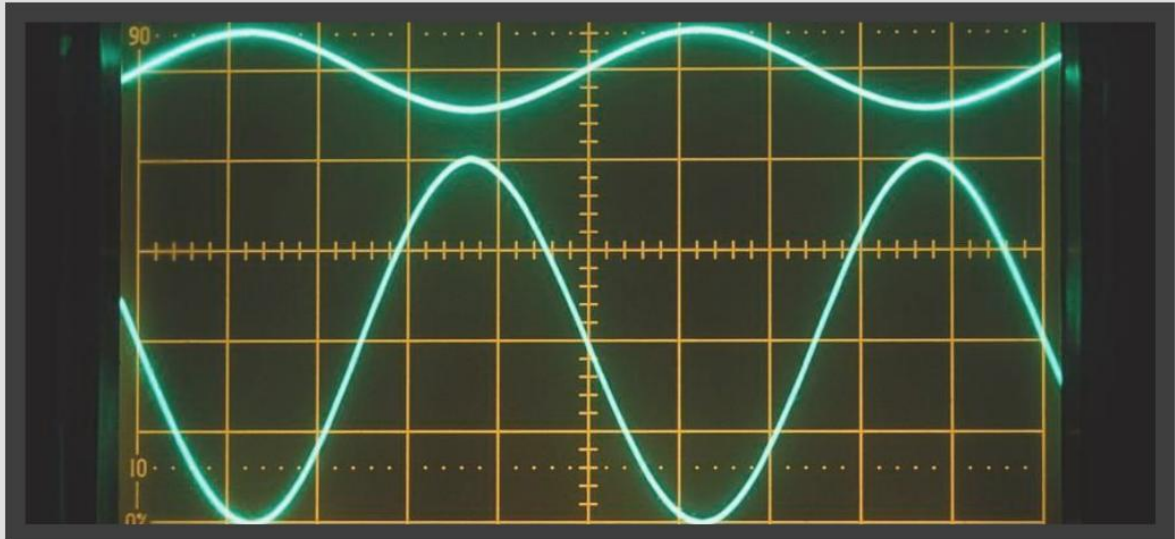




Fakultas
Teknologi
Informasi
Universitas
Andalas



LABORATORIUM SINYAL & SISTEM



MODUL PRAKTIKUM PENGOLAHAN SISTEM DIGITAL

07 /

Signals and Systems
Laboratory
Computer Engineering
Department
Andalas University
2025

PENERAPAN FILTER DIGITAL DENGAN SINYAL EKSTERNAL

1. CPMK

1. Mahasiswa mampu melaksanakan eksperimen untuk menjelaskan konsep dasar pengolahan sinyal melalui simulasi perangkat lunak dan modul perangkat keras (CP-3, PI-1)
2. Mahasiswa mampu menganalisis dan menginterpretasikan sinyal dalam domain waktu dan frekuensi.(CP-3, PI-1)
3. Mahasiswa mampu merancang dan mengevaluasi sistem filter digital sederhana. (CP-3, PI-3)
4. Mahasiswa mampu menyusun laporan hasil praktikum dengan sistematika dan bahasa ilmiah yang baik. (CP-6, PI-2)

SUB CPMK

- a) Menganalisis sinyal pada domain waktu dan domain frekuensi menggunakan Fast Fourier Transform (FFT).
- b) Menerapkan filter digital Butterworth IIR orde 2 berupa Low Pass Filter, High Pass Filter, Band Pass Filter, dan Band Stop Filter pada sinyal digital.
- c) Mengidentifikasi dan membedakan komponen sinyal informasi serta komponen noise berdasarkan spektrum frekuensi yang diperoleh.

2. DASAR TEORI

Tujuan dari penerapan filter digital adalah untuk menghasilkan suatu sistem yang mampu memberikan perlakuan tertentu terhadap sinyal input sesuai dengan karakteristik frekuensi yang diinginkan. Dalam praktikum ini, filter digunakan untuk memisahkan sinyal informasi dengan komponen noise berdasarkan analisis spektrum frekuensinya. Sebagai contoh, komponen frekuensi tertentu yang dianggap sebagai noise dapat direduksi sehingga sinyal utama menjadi lebih jelas.

Sinyal yang digunakan pada praktikum diperoleh dari sumber eksternal berupa generator sinyal, yang menghasilkan sinyal analog. Sinyal tersebut kemudian dikonversi menjadi sinyal digital menggunakan Analog to Digital Converter (ADC) dengan frekuensi sampling sebesar 2000 Hz. Proses ini dilakukan melalui fitur ADC to RAM, dimana data hasil sampling disimpan ke dalam

memori (RAM), kemudian diambil kembali menggunakan GET DATA RAM untuk dianalisis lebih lanjut.

2.1 Penggabungan Sinyal dan Noise

Dalam sistem elektronik nyata, sinyal informasi yang berjalan melalui kabel atau komponen hardware tidak pernah berjalan sendiri. Ia selalu bercampur dengan gangguan (*noise*) yang berasal dari berbagai sumber, seperti interferensi elektromagnetik dari perangkat di sekitarnya, osilasi internal komponen, atau bocoran antar jalur kabel (*crosstalk*). Hasil campuran ini adalah sinyal gabungan yang jika ditampilkan di osiloskop terlihat rumit dan tidak beraturan, padahal di dalamnya terkandung komponen-komponen frekuensi yang terpisah.

Pada praktikum ini, kondisi tersebut disimulasikan secara terkontrol menggunakan tiga generator sinyal:

- Generator 1 menghasilkan sinyal sinus 25 Hz, 2,5 Vpp (dari NI myDAQ atau Arduino), yang berperan sebagai sinyal informasi utama.
- Generator 2 menghasilkan sinyal sinus 100 Hz, 300 mV, sebagai sinyal informasi sekunder.
- Generator 3 menghasilkan sinyal sinus 800 Hz, 100 mV, sebagai komponen frekuensi tinggi atau noise.

Ketiga sinyal ini dijumlahkan oleh rangkaian internal modul RSA09, menghasilkan satu sinyal campuran yang kemudian masuk ke ADC untuk disampling dan dianalisis.

2.2 Analisis Spektrum Frekuensi menggunakan FFT

Osiloskop menampilkan sinyal dalam **domain waktu**, yaitu grafik tegangan terhadap waktu. Pada domain ini, sinyal gabungan dari tiga frekuensi berbeda akan terlihat sebagai satu gelombang yang tidak beraturan, sehingga sulit untuk mengidentifikasi komponen penyusunnya secara langsung.

Untuk keperluan tersebut, digunakan *Fast Fourier Transform* (FFT), yaitu algoritma komputasi efisien untuk menghitung *Discrete Fourier Transform* (DFT). FFT mengubah representasi sinyal dari domain waktu ke domain frekuensi, sehingga setiap komponen frekuensi yang menyusun sinyal dapat terlihat secara individual. Secara matematis, DFT didefinisikan sebagai:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}$$

Di mana:

$x[n]$: sinyal dalam domain waktu

$X[k]$: representasi sinyal dalam domain frekuensi

N : jumlah sampel

k : indeks frekuensi

Hasil FFT ditampilkan sebagai grafik dengan sumbu horizontal berupa frekuensi (Hz) dan sumbu vertikal berupa amplitudo atau magnitudo. Pada sinyal gabungan praktikum ini, grafik FFT akan menampilkan tiga puncak (*peak*) yang tajam pada frekuensi 25 Hz, 100 Hz, dan 800 Hz — masing-masing merepresentasikan satu komponen sinyal penyusun. Analisis ini menjadi dasar untuk menentukan jenis dan parameter filter yang akan digunakan.

Frekuensi sampling yang digunakan pada praktikum ini adalah 2000 Hz. Sesuai Teorema Nyquist, frekuensi maksimum yang dapat dideteksi secara akurat adalah setengah dari frekuensi sampling, yaitu 1000 Hz. Komponen 800 Hz masih berada dalam batas ini sehingga dapat terdeteksi dengan baik.

2.3 Filter Digital Butterworth

Filter digital adalah sistem yang meloloskan komponen frekuensi tertentu sambil meredam komponen frekuensi lainnya. Pada praktikum ini digunakan filter Butterworth IIR (*Infinite Impulse Response*) orde 2, yang memiliki karakteristik respons magnitudo paling datar (*maximally flat*) di dalam pita frekuensi yang diloloskan, tanpa riak (*ripple*). Filter IIR bekerja dengan umpan balik, sehingga outputnya dipengaruhi oleh input saat ini, input sebelumnya, dan output sebelumnya, sesuai persamaan beda:

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{i=1}^N a_i y[n-i]$$

Terdapat empat tipe filter yang digunakan dalam praktikum ini:

a. Low Pass Filter (LPF)

LPF meloloskan komponen frekuensi yang berada di bawah frekuensi cutoff (f_c) dan meredam komponen di atasnya. Pada praktikum ini, LPF digunakan untuk mengisolasi komponen 25 Hz dengan menetapkan f_c di antara 25 Hz dan 100 Hz. Komponen 100 Hz dan 800 Hz akan ikut teredam, sehingga hanya sinyal 25 Hz yang tersisa pada output.

b. High Pass Filter (HPF)

HPF meloloskan komponen frekuensi yang berada di atas frekuensi cutoff dan meredam komponen di bawahnya. Pada praktikum ini, HPF digunakan untuk mengisolasi komponen 800 Hz dengan menetapkan f_c di antara 100 Hz dan 800 Hz. Komponen 25 Hz dan 100 Hz akan teredam, sehingga hanya sinyal 800 Hz yang tersisa.

c. Band Pass Filter (BPF)

BPF meloloskan komponen frekuensi yang berada dalam suatu rentang tertentu antara frekuensi cutoff bawah (f_{c1}) dan frekuensi cutoff atas (f_{c2}), serta meredam komponen di luar rentang tersebut. Pada praktikum ini, BPF digunakan untuk mengisolasi komponen

100 Hz dengan menetapkan rentang pass-band yang mencakup 100 Hz namun tidak mencakup 25 Hz maupun 800 Hz.

d. Band Stop Filter (BSF)

BSF, yang juga dikenal sebagai *notch filter*, bekerja kebalikan dari BPF — ia meredam komponen frekuensi dalam suatu rentang tertentu dan meloloskan komponen di luar rentang tersebut. Filter ini digunakan ketika noise berada pada suatu pita frekuensi yang sempit dan dapat diidentifikasi, sementara sinyal informasi berada di luar pita tersebut. Pada praktikum ini, BSF dapat digunakan untuk meredam komponen 100 Hz tanpa mengganggu komponen 25 Hz maupun 800 Hz.

2.4 Proses Akuisisi Sinyal dan Fungsi Potensiometer OFFSET

Sinyal dari generator bersifat analog, sehingga perlu dikonversi terlebih dahulu menjadi sinyal digital sebelum dapat diproses oleh perangkat lunak. Proses ini dilakukan oleh ADC (*Analog to Digital Converter*) yang terdapat pada modul RSA09 dengan frekuensi sampling 2000 Hz. Data hasil sampling disimpan ke dalam memori (*ADC to RAM*), kemudian diambil kembali melalui fitur *GET DATA RAM* untuk ditampilkan dan dianalisis.

Sebelum proses sampling dilakukan, posisi sinyal perlu disesuaikan menggunakan Potensiometer OFFSET pada modul RSA09. ADC hanya dapat membaca tegangan dalam rentang tertentu. Jika sinyal analog memiliki tegangan DC yang terlalu rendah atau terlalu tinggi, sebagian gelombang akan berada di luar rentang baca ADC dan terpotong (*clipping*), mengakibatkan data yang terekam tidak lengkap dan merusak hasil analisis FFT maupun filtering.

Potensiometer OFFSET berfungsi menambahkan atau mengurangi tegangan DC pada sinyal sehingga posisinya bergeser ke titik yang sesuai dengan rentang baca ADC. Pengaturan ini dilakukan dengan mengamati sinyal pada osiloskop dan memastikan sinyal berada tepat di atas *baseline* sebelum proses sampling dimulai.

3. ALAT DAN BAHAN

1. Modul RS A09 *Digital Signal Processing*
2. Laptop
3. Osiloskop
4. NI myDAQ
5. Arduino Uno

4. PROSEDUR PRAKTIKUM

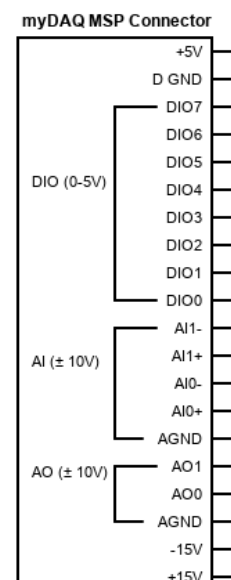
4.1 Percobaan 1 : Pemfilteran Sinyal Eksternal Menggunakan NI myDAQ

1. Nyalakan modul RSA09, NI myDAQ, osiloskop, dan komputer.
2. Hubungkan probe CH1 osiloskop ke TEST POINT "IN" dan ground probe ke TEST POINT "GND".
3. Amati tampilan sinyal pada osiloskop. Jika posisi sinyal masih berada di tengah baseline, atur potensiometer OFFSET pada modul RSA09 hingga sinyal berada tepat di atas baseline.



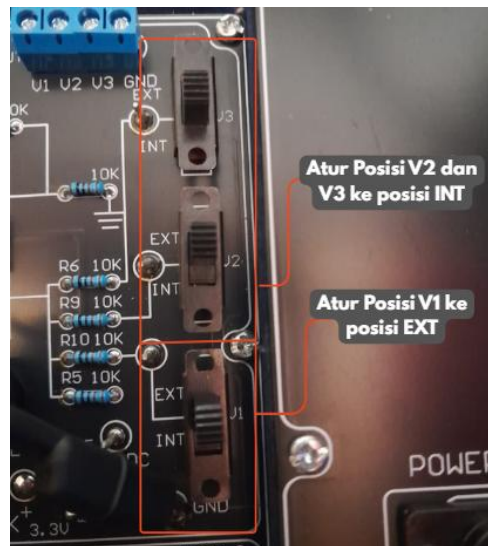
Gambar 4.1 Potensiometer OFFSET pada Modul RSA09

4. Hubungkan NI myDAQ ke komputer menggunakan kabel USB.
5. Buka perangkat lunak NI ELVISmx Function Generator.
6. Hubungkan terminal AO0 NI myDAQ ke terminal EXT pada Generator Sinyal 1 modul RSA09 dan hubungkan AGND NI myDAQ ke GND modul RSA09.



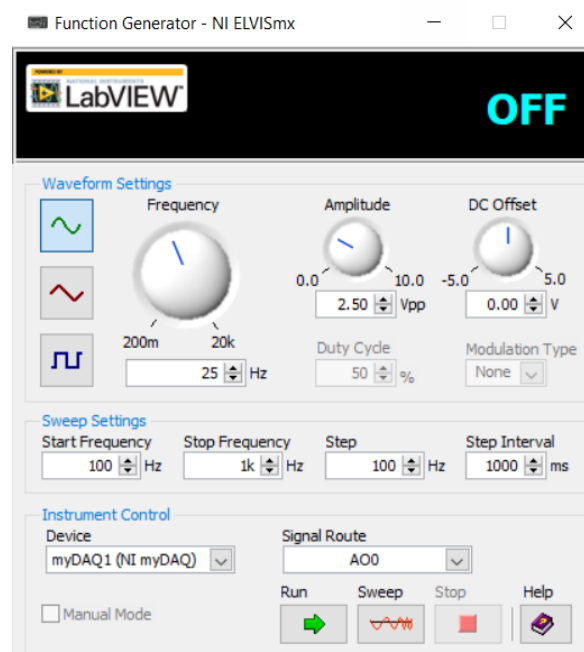
Gambar 4.2 Pinout NI myDAQ

7. Atur switch V1 pada posisi EXT, sedangkan switch V2 dan V3 pada posisi INT.



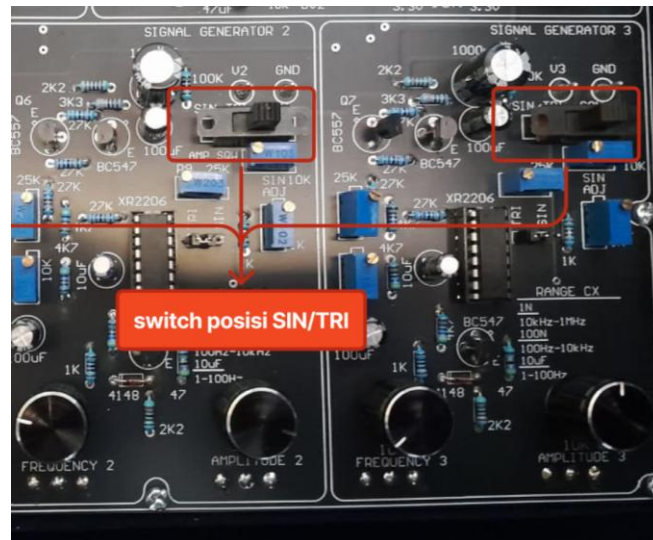
Gambar 4.3 Posisi Switch V1, V2, dan V3

8. Atur **NI ELVISmx Function Generator** dengan parameter:
- Bentuk gelombang : Sinus
 - Frekuensi : 25 Hz
 - Amplitudo : 2.5 Vpp



Gambar 4.4 Konfigurasi Function Generator NI myDAQ

9. Atur **Generator Sinyal 2** pada modul RSA09 sehingga menghasilkan sinyal sinus dengan frekuensi **100 Hz** dan amplitudo **300 mV**.
10. Atur **Generator Sinyal 3** pada modul RSA09 sehingga menghasilkan sinyal sinus dengan frekuensi **800 Hz** dan amplitudo **100 mV**.
11. Pastikan switch generator berada pada posisi SIN/TRI dan jumper IC2206 berada pada posisi SIN.

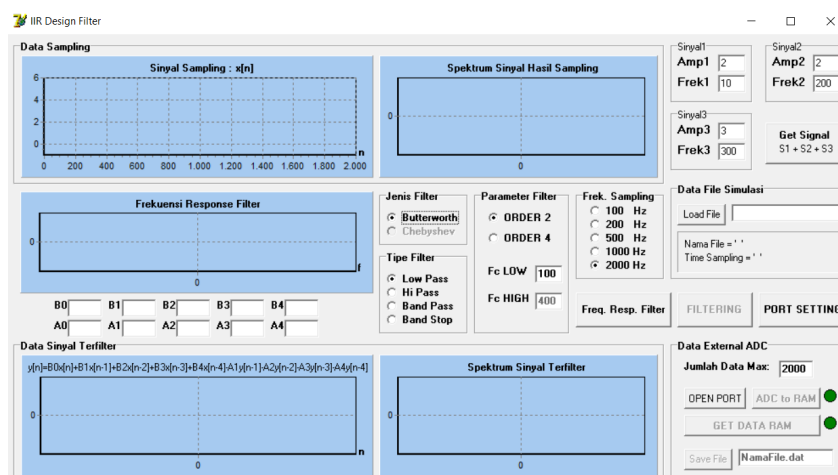


Gambar 4.5 Posisi Switch SIN/TRI



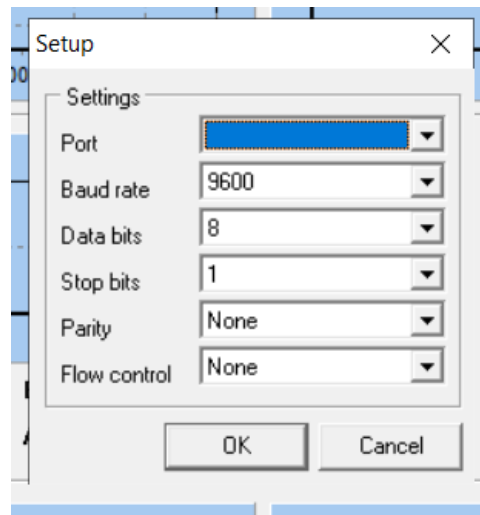
Gambar 4.6 Posisi Jumper SIN

12. Hubungkan modul RSA09 ke komputer menggunakan kabel USB.
13. Gunakan Device Manager untuk mengetahui nomor COM Port yang digunakan.
14. Jalankan perangkat lunak DigitalFilter.exe.



Gambar 4.7 Tampilan Awal DigitalFilter.exe

15. Klik tombol PORT SETTING kemudian pilih COM Port yang sesuai.



Gambar 4.8 Pengaturan COM Port

16. Klik tombol OPEN PORT hingga berubah menjadi CLOSE PORT.
17. Pilih frekuensi sampling sebesar 2000 Hz.
18. Klik tombol ADC to RAM dan tunggu hingga proses sampling selesai.
19. Klik tombol GET DATA RAM untuk menampilkan sinyal hasil sampling.
20. Amati dan catat bentuk sinyal pada grafik Sinyal Sampling dan Spektrum Sinyal Hasil Sampling.
21. Pilih filter Butterworth Orde 2 dengan tipe Low Pass Filter.
22. Atur frekuensi cutoff sehingga komponen frekuensi **25 Hz** dapat dipisahkan dari sinyal gabungan.
23. Amati dan catat bentuk sinyal keluaran serta spektrum frekuensinya.
24. Ulangi langkah 21–23 menggunakan High Pass Filter untuk memperoleh komponen frekuensi **800 Hz**.
25. Ulangi langkah 21–23 menggunakan Band Pass Filter untuk memperoleh komponen frekuensi **100 Hz**.
26. Ulangi langkah 21–23 menggunakan Band Stop Filter untuk meredam komponen frekuensi 100 Hz dan mengamati komponen frekuensi yang tersisa.
27. Simpan seluruh data hasil pengamatan untuk digunakan pada analisis dan pembahasan.

4.2 Percobaan 2 : Penyaringan Noise Menggunakan Arduino Uno

1. Nyalakan modul RSA09, Arduino Uno, osiloskop, dan komputer.
2. Hubungkan probe CH1 osiloskop ke TEST POINT "IN" dan ground probe ke TEST POINT "GND".

3. Amati tampilan sinyal pada osiloskop. Jika posisi sinyal masih berada di tengah baseline, atur potensiometer OFFSET pada modul RSA09 hingga sinyal berada tepat di atas baseline.
4. Hubungkan Arduino Uno ke komputer menggunakan kabel USB.
5. Jalankan perangkat lunak Arduino IDE.
6. Buat sketch baru dan masukkan program berikut.

```
const byte OUTPUT_PIN = 9;
const int noiseFreq[] = { 250, 275, 300, 325, 350, 375, 400, 425, 450, 475, 500, 525, 550 };
const int N_FREQ = sizeof(noiseFreq) / sizeof(noiseFreq[0]);

unsigned long lastChange = 0;
int indexFreq = 0;

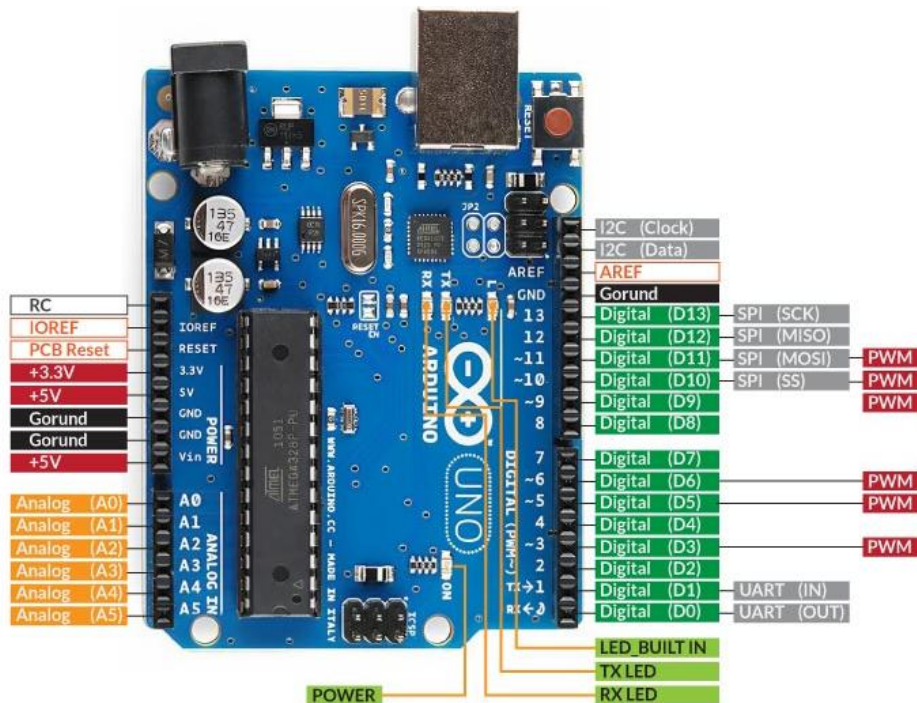
void setup() {
  Serial.begin(115200);
  pinMode(OUTPUT_PIN, OUTPUT);
  tone(OUTPUT_PIN, noiseFreq[indexFreq]);
}

void loop() {
  if (millis() - lastChange >= 50) {
    lastChange = millis();
    indexFreq++;

    if (indexFreq >= N_FREQ) indexFreq = 0;

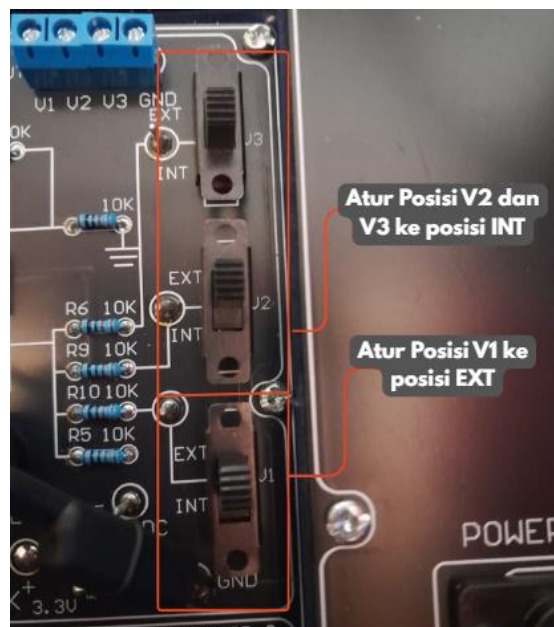
    tone(OUTPUT_PIN, noiseFreq[indexFreq]);
    Serial.print("Noise Frequency = ");
    Serial.print(noiseFreq[indexFreq]);
    Serial.println(" Hz");
  }
}
```

7. Kompilasi program dan unggah program ke Arduino Uno.
8. Hubungkan pin D9 Arduino Uno ke terminal EXT pada Generator Sinyal 1 modul RSA09.
9. Hubungkan pin GND Arduino Uno ke terminal GND modul RSA09.



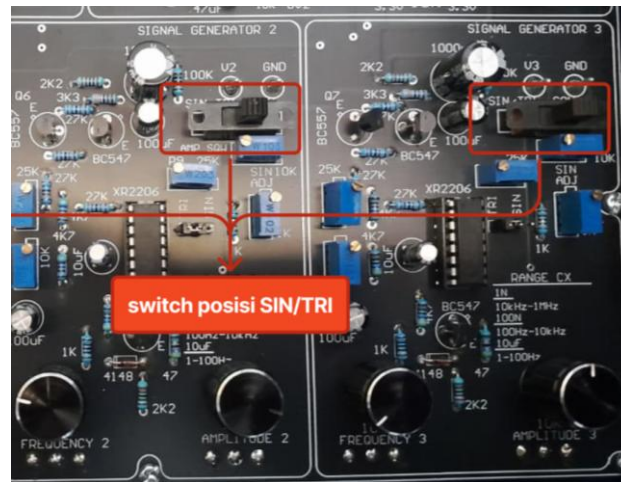
Gambar 4.9 Pinout Arduino Uno

10. Atur switch V1 pada posisi EXT, sedangkan switch V2 dan V3 pada posisi INT.

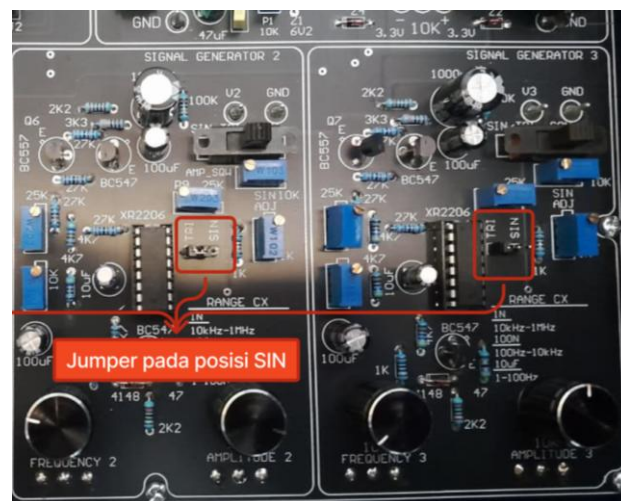


Gambar 4.10 Posisi Switch V1, V2, dan V3

11. Atur **Generator Sinyal 2** pada modul RSA09 sehingga menghasilkan sinyal sinus dengan frekuensi **100 Hz** dan amplitudo **300 mV**.
12. Atur **Generator Sinyal 3** pada modul RSA09 sehingga menghasilkan sinyal sinus dengan frekuensi **800 Hz** dan amplitudo **100 mV**.
13. Pastikan switch generator berada pada posisi SIN/TRI dan jumper IC2206 berada pada posisi SIN.

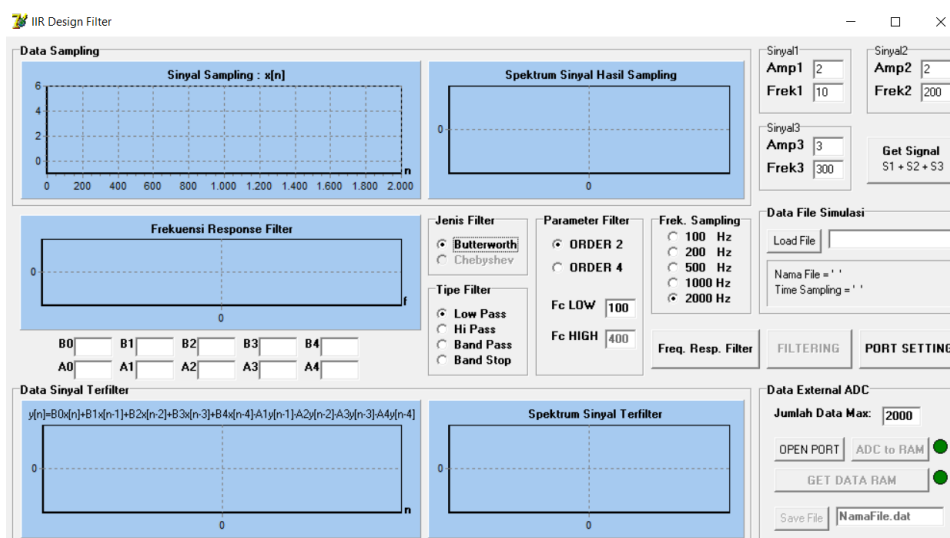


Gambar 4.11 Posisi Switch SIN/TRI



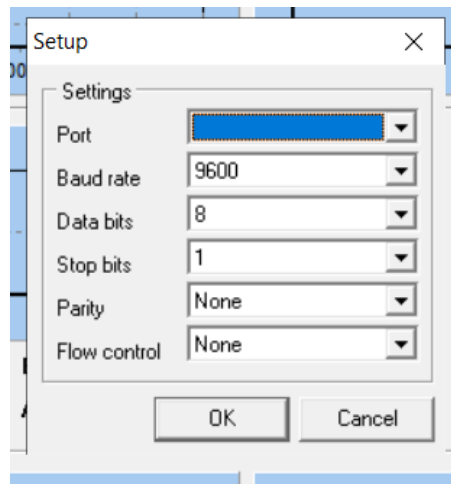
Gambar 4.12 Posisi Jumper SIN

14. Hubungkan modul RSA09 ke komputer menggunakan kabel USB.
15. Gunakan Device Manager untuk mengetahui nomor COM Port yang digunakan oleh modul RSA09.
16. Jalankan perangkat lunak DigitalFilter.exe.



Gambar 4.13 Tampilan Awal DigitalFilter.exe

17. Klik tombol PORT SETTING kemudian pilih COM Port yang sesuai.



Gambar 4.14 Pengaturan COM Port

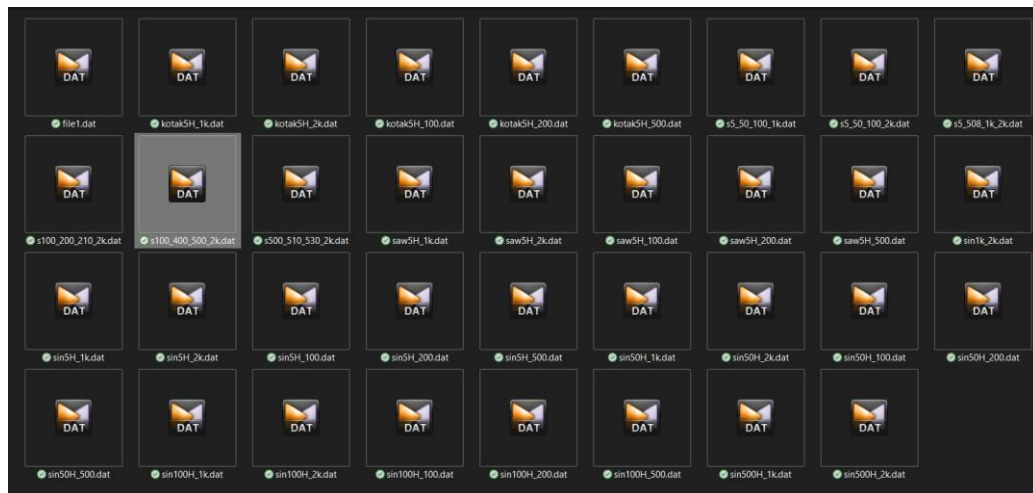
18. Klik tombol OPEN PORT hingga berubah menjadi CLOSE PORT.
19. Pilih frekuensi sampling sebesar 2000 Hz.
20. Klik tombol ADC to RAM dan tunggu hingga proses sampling selesai.
21. Klik tombol GET DATA RAM untuk menampilkan sinyal hasil sampling.
22. Amati dan catat bentuk sinyal pada grafik Sinyal Sampling dan Spektrum Sinyal Hasil Sampling.
23. Identifikasi komponen frekuensi sinyal informasi dan komponen frekuensi noise yang muncul pada spektrum frekuensi.
24. Pilih filter Butterworth Orde 2 dengan tipe **Low Pass Filter**.
25. Atur frekuensi cutoff yang sesuai, kemudian amati dan catat hasil pemfilteran.
26. Ulangi langkah 24-25 menggunakan **High Pass Filter**.
27. Ulangi langkah 24-25 menggunakan **Band Stop Filter**.
28. Bandingkan hasil yang diperoleh dari masing-masing jenis filter.
29. Tentukan jenis filter yang paling efektif untuk mempertahankan komponen frekuensi 100 Hz dan 800 Hz serta meredam komponen noise pada rentang 250 Hz hingga 550 Hz.
30. Simpan seluruh data hasil pengamatan untuk digunakan pada analisis dan pembahasan.

4.3 Percobaan 3 : Analisis dan Pemfilteran Data Tersimpan

1. Nyalakan komputer dan jalankan perangkat lunak DigitalFilter.exe.
2. Klik tombol LOAD FILE.
3. Pilih file data (.dat) pada folder dat :

s100_400_500_2k.dat

4. Klik Open untuk memuat data ke dalam perangkat lunak.



Gambar 4.15 Pemilihan File Data

5. Amati grafik Sinyal Sampling dan Spektrum Sinyal Hasil Sampling yang ditampilkan.
6. Identifikasi komponen frekuensi yang muncul pada spektrum frekuensi.
7. Catat frekuensi-frekuensi dominan yang terdeteksi pada data.
8. Pilih filter Butterworth Orde 2 dengan tipe Low Pass Filter.
9. Tentukan nilai frekuensi cutoff yang sesuai, kemudian klik tombol FILTERING.
10. Amati dan catat bentuk sinyal hasil filter serta spektrum frekuensi yang diperoleh.
11. Ulangi langkah 8–10 menggunakan **High Pass Filter**.
12. Ulangi langkah 8–10 menggunakan **Band Pass Filter**.
13. Ulangi langkah 8–10 menggunakan **Band Stop Filter**.
14. Bandingkan hasil yang diperoleh dari Low Pass Filter, High Pass Filter, Band Pass Filter, dan Band Stop Filter.
15. Tentukan parameter filter yang paling sesuai untuk memperoleh komponen frekuensi yang diinginkan.
16. Simpan seluruh data hasil pengamatan untuk digunakan pada analisis dan pembahasan.

5. TUGAS PADA LAPORAN PRAKTIKUM

1. Jenis filter manakah yang paling efektif untuk tujuan percobaan yang dilakukan? Jelaskan berdasarkan hasil spektrum frekuensi dan sinyal keluaran yang diperoleh.
2. Mengapa analisis pada domain frekuensi (FFT) diperlukan sebelum menentukan jenis filter yang akan digunakan?
3. Apa kesimpulan yang Anda peroleh dari praktikum ini?